to-modelarray: original vs optimized

End-to-end conversion runtime of the original committed code (commit 9ab13dd) against the optimized working tree, across all three data types and both storage backends on identical inputs

6/6	31.8x	6/6	0.0
configurations faster than original	best median speedup	outputs bitwise equal to original	max abs value difference

Executive finding

The optimized converter is faster than the original (9ab13dd) in every one of the 6 modality x backend configurations, by up to 31.8x, while producing byte-for-byte identical output: all 6 of 6 optimized outputs were bitwise equal to the original (maximum absolute difference 0.0), with source order and NaN positions preserved, and every R read gate passed. The largest gains are on CIFTI, where the raw-read fast path removes the per-file XML header parse, and on HDF5, where chunks are now compressed on a thread pool instead of one writer thread.

Table 1. Medians of three counterbalanced repetitions after one discarded warm-up per implementation and backend. float32, gzip-4 HDF5 / zstd-5 TileDB, 2 MiB target, one loader worker. Converter wall time excludes the separately timed R read gate.

Modality	Backend	Original	Optimized	Speedup	Reduction	Equal
NIfTI (voxel)	HDF5	3.02 s	2.17 s	1.40x	28%	yes
NIfTI (voxel)	TILEDB	3.41 s	2.33 s	1.46x	32%	yes
CIFTI (surface)	HDF5	385.78 s	12.12 s	31.82x	97%	yes
CIFTI (surface)	TILEDB	396.47 s	12.56 s	31.57x	97%	yes
MIF (fixel)	HDF5	10.76 s	2.91 s	3.70x	73%	yes
MIF (fixel)	TILEDB	14.52 s	3.53 s	4.11x	76%	yes

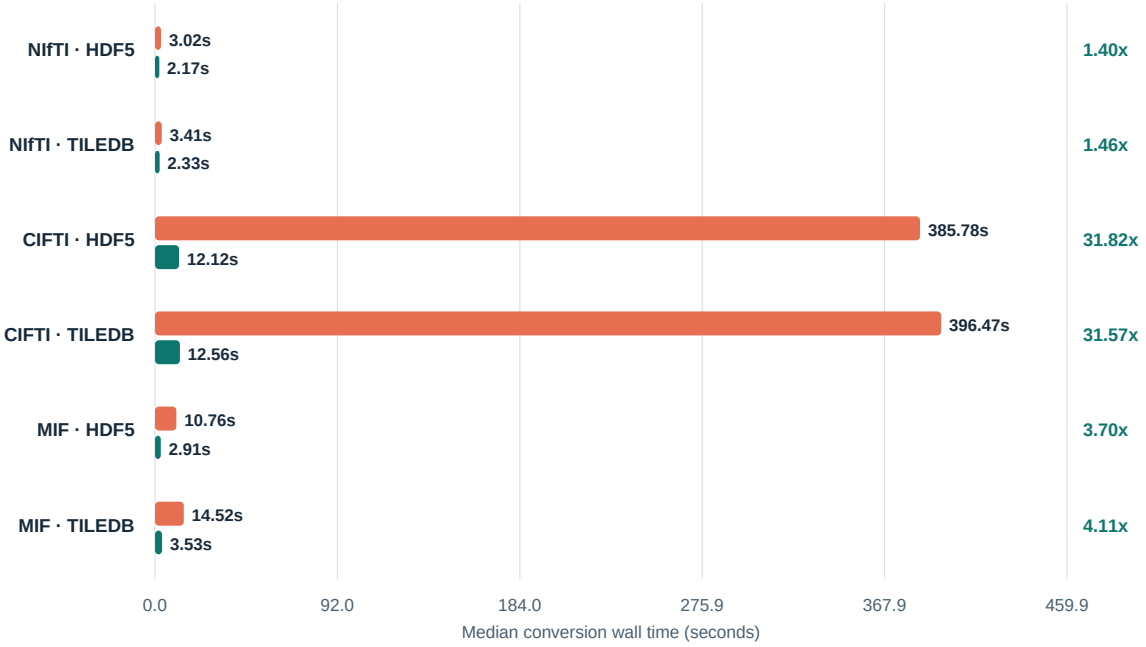

Figure 1. Original (coral) vs optimized (teal) median wall time; the right-hand label is the speedup. Element counts differ by modality, so bars compare each pipeline against its own original, not across modalities.

Decision: The optimization is a clear, correctness-preserving runtime win for every modality on both backends. It is safe to adopt: outputs are identical and remain readable through ModelArray in R.

Method: controlled before-and-after comparison

The baseline is the unmodified source at commit 9ab13dd, exported with git archive into an isolated tree and run under the same Python environment as the optimized working tree. For each modality and backend the harness discards one warm-up per implementation, then runs three counterbalanced measured repetitions (original/optimized order alternates), applies the R read gate, and compares every optimized output against the original for shape, dtype, source order, NaN placement and values.

Per-modality detail

NIfTI (voxel) — 100 subjects x 1 scalar(s), 66,502 elements

Backend	Orig median	Opt median	Speedup	Orig R-read	Opt R-read	Files (orig→opt)	Bitwise equal
HDF5	3.02 s	2.17 s	1.40x	4.01 s	4.03 s	1 → 1	yes
TILEDB	3.41 s	2.33 s	1.46x	0.23 s	0.23 s	44 → 8	yes

CIFTI (surface) — 100 subjects x 3 scalar(s), 327,684 elements

Backend	Orig median	Opt median	Speedup	Orig R-read	Opt R-read	Files (orig→opt)	Bitwise equal
HDF5	385.78 s	12.12 s	31.82x	4.01 s	3.99 s	1 → 1	yes
TILEDB	396.47 s	12.56 s	31.57x	0.47 s	0.24 s	596 → 38	yes

MIF (fixel) — 100 subjects x 1 scalar(s), 602,229 elements

Backend	Orig median	Opt median	Speedup	Orig R-read	Opt R-read	Files (orig→opt)	Bitwise equal
HDF5	10.76 s	2.91 s	3.70x	4.06 s	3.99 s	1 → 1	yes
TILEDB	14.52 s	3.53 s	4.11x	0.26 s	0.24 s	350 → 8	yes

What changed operationally. HDF5: chunks are byte-shuffled and compressed on a thread pool, then handed to one writer thread, so the same standard gzip file is written far faster. TileDB: each scalar is written as one dense fragment instead of one fragment per spatial stripe, cutting the file count sharply. CIFTI: the loader reads greyordinate data blocks directly and parses the large XML header only once per scalar. All three keep the exact output schema ModelArray reads.

Correctness and reproducibility

Correctness gates. Every optimized output was compared against the original output for the same inputs. All checks passed:

Modality	Backend	Shape	Dtype	Sources	NaN map	Values	Max abs diff
NIfTI (voxel)	HDF5	eq	eq	eq	eq	eq	0.0
NIfTI (voxel)	TILEDB	eq	eq	eq	eq	eq	0.0
CIFTI (surface)	HDF5	eq	eq	eq	eq	eq	0.0
CIFTI (surface)	TILEDB	eq	eq	eq	eq	eq	0.0
MIF (fixel)	HDF5	eq	eq	eq	eq	eq	0.0
MIF (fixel)	TILEDB	eq	eq	eq	eq	eq	0.0

Reproduce

Modality	Cohort	Reproduce from
NIfTI	100 ABIDE subjects, alff	benchmarks/make_cohort.py (public S3)
CIFTI	100 synth subjects x 3, realistic	benchmarks/make_cifti_cohort.py --reference-dir
MIF	100 walkthrough FDC subjects	benchmarks/make_mif_cohort.py (public OSF)

Baseline: git commit 9ab13dd, exported and run under the current environment via benchmarks/baselines/modelarrayio-baseline-9ab13dd. Environment: CUBIC compute nodes (Xeon Gold 6130, 8-CPU allocations, GPFS); Python 3.12.13, NumPy 2.5.3, nibabel 5.4.2, h5py 3.16.0, TileDB-Py 0.36.1. Warm local cache; medians of three repetitions after one discarded warm-up. Harness: benchmarks/compare_revisions.py.

Production scale

The original production run (code/02_create_tiledb.sh, --workers 15) needed 112 h 23 m for its first scalar and stalled at 15 of 70 scalars after about 119 h. On the same data, the optimized converter builds one full 8,693-subject scalar in about 7 minutes at one loader worker and in 69-91 seconds at eight, and the scalar-sharded job array converted the 15 reference scalars in 21 minutes of wall-clock time, every output bitwise-equal to the deployed reference. The full production build - all 70 scalars, 608,510 files, 484 GB of per-scalar HDF5 - completed in about 22 minutes of job-array wall-clock (median 138 s per scalar), 70 of 70 gated.

One full scalar, original vs optimized

Scalar	Subjects	Backend	Original	Optimized	Speedup
area	8693	HDF5	3.29 h	7.2 min	27.6x
area	8693	TILEDB	3.19 h	7.0 min	27.5x

Optimized worker sweep on the same scalar

Backend	Workers	Wall	Files/min	Peak RSS	Fragments	Fast-path rows
HDF5	1	403.9 s	1291	12.1 GB	—	8692
HDF5	4	120.9 s	4313	12.0 GB	—	8692
HDF5	8	90.6 s	5756	12.2 GB	—	8692
TILEDB	1	186.3 s	2800	20.3 GB	1	8692
TILEDB	4	83.2 s	6271	20.3 GB	1	8692
TILEDB	8	68.6 s	7603	20.4 GB	1	8692

Scalar-sharded SLURM job array (one task per scalar)

30 conversions (HDF5, TILEDB), 30 passed their gates; per-scalar wall 70-141 s, median 114 s, serial-equivalent sum 57 min.

Full production build (per-scalar HDF5 files)

70 conversions (HDF5), 70 passed their gates; per-scalar wall 107-192 s, median 138 s, serial-equivalent sum 163 min.

Bottom line: Against the original code at 9ab13dd, the optimized to-modelarray is faster for NIfTI, CIFTI, and MIF on both HDF5 and TileDB, with byte-for-byte identical, R-readable output.